Inheritance and Polymorphism in Python OOP throught a geometry example

Abdesselam Filali
infos@filali.net
https://github.com/Abdou-fi/python_libraries

Mai 30, 2026

Python is a multiparadigm programming language that supports object-oriented programming (OOP) as well as procedural and functional programming. OOP is a programming paradigm that uses objects and classes to structure code in a way that is more intuitive and easier to manage.

1 What is OOP?

OOP stands for Object-Oriented Programming.

Python is an object-oriented language, allowing you to structure your code using classes and objects for better organization and reusability.

2 Advantages of OOP

- Provides a clear structure to programs
- Makes code easier to maintain, reuse, and debug
- Helps keep your code DRY (Don't Repeat Yourself)
- Allows you to build reusable applications with less code

Tip: The DRY principle means you should avoid writing the same code more than once. Move repeated code into functions or classes and reuse it.

3 What are Classes and Objects?

Classes and objects are the two core concepts in object-oriented programming.

A class defines what an object should look like, and an object is created based on that class.

Otherwise : a class is a blueprint for creating objects (a particular data structure), an object is an instance of a class.

For example:

Class	**Objects**
Fruit	Apple, Banana, Mango
Car	Volvo, Audi, Toyota

When you create an object from a class, it inherits all the variables and functions defined inside that class.

The main concepts are :

- Classes and objects
- The **init**() method
- The self parameter
- Properties and methods
- Inheritance and polymorphism
- Encapsulation and inner classes (will be treated in a further article)

3.1 The init() Method

The **init**() method is a special method in Python classes, known as a constructor. It is automatically called when a new object is created from a class and allows you to initialize the object's attributes.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Amine", 36)
```

3.2 The self Parameter

The self parameter is a reference to the current instance of the class and is used to access variables that belong to the class.

It does not have to be named self , you can call it whatever you like, but it has to be the first parameter of any function in the class:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("Amine", 36)
p1.myfunc()
```

3.3 Properties and Methods

A property is a special type of attribute that allows you to define a method as if it were an attribute. This means you can access the method like an attribute, without needing to use parentheses.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
    def myfunc(self):
        return "Hello my name is " + self.name

p1 = Person("Amine", 36)
print(p1.myfunc)
```

A method is a function that is defined inside a class and is used to perform operations on the object.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("Amine", 36)
p1.myfunc()
```

3.4 Inheritance and Polymorphism

3.4.1 Inheritance

Inheritance, a powerful feature in object-oriented programming, is the concept of creating a new class that is a modified version of an existing class. The new class inherits all the attributes and methods of the existing class, and you can add new attributes and methods or override existing ones.

It allows allows a new class (called a child or subclass) to inherit attributes and methods from an existing class (called a parent or superclass). This promotes code reusability and establishes a natural hierarchical relationship between classes.

```
class Person:

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def informations(self):
        print("Hello my name is " + self.name)

class Student(Person):
    def __init__(self, name, age, grade):
        super().__init__(name, age)
        self.grade = grade

    def informations(self):
        print("Hello my name is " + self.name + " and I am in grade " + str(self.grade))

p1 = Student("Amine", 36, 12)
```

```
p1.informations()
```

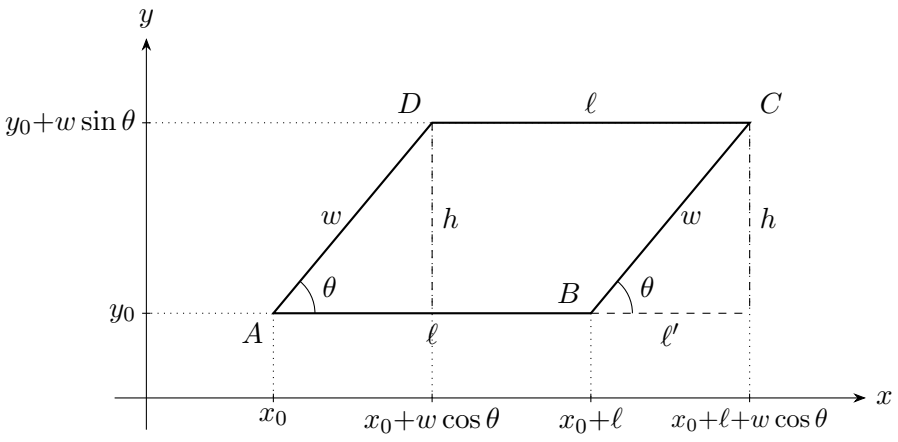
3.4.2 polymorphism

Polymorphism allows you to use a unified interface to access different types of objects. This means that you can use the same method name to perform different tasks based on the object that is calling the method.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def myfunc(self):
        print("Hello my name is " + self.name)

class Student(Person):
    def __init__(self, name, age, grade):
        super().__init__(name, age)
        self.grade = grade

    def myfunc(self):
        print("Hello my name is " + self.name + " and I am in grade " + str(self.grade))

p1 = Person("Amine", 36)
p2 = Student("Sarah", 12, 10)

p1.myfunc()
p2.myfunc()
```

4 Example of Inheritance & Polymorphism

The following representation is a parallelogram with sides ℓ and w and height h (the distance between the two parallel sides of length ℓ). You find the relations between the angle θ, the length and the width of the parallelogram translated into vertex coordinates.



Relations between ℓ', h and w:

$$\cos\theta = \frac{\ell'}{w} \quad \Rightarrow \quad \ell' = w\cos\theta$$
$$\sin\theta = \frac{h}{w} \quad \Rightarrow \quad h = w\sin\theta$$

Vertex coordinates:

$$A = (x_0,\ y_0)$$
$$B = (x_0 + \ell,\ y_0)$$
$$C = (x_0 + \ell + w\cos\theta,\ y_0 + w\sin\theta)$$
$$D = (x_0 + w\cos\theta,\ y_0 + w\sin\theta)$$

The parallelogram is completely defined by the parameters x_0, y_0, ℓ, w, and θ. The coordinates of the vertices A, B, C, and D can be calculated using these parameters.

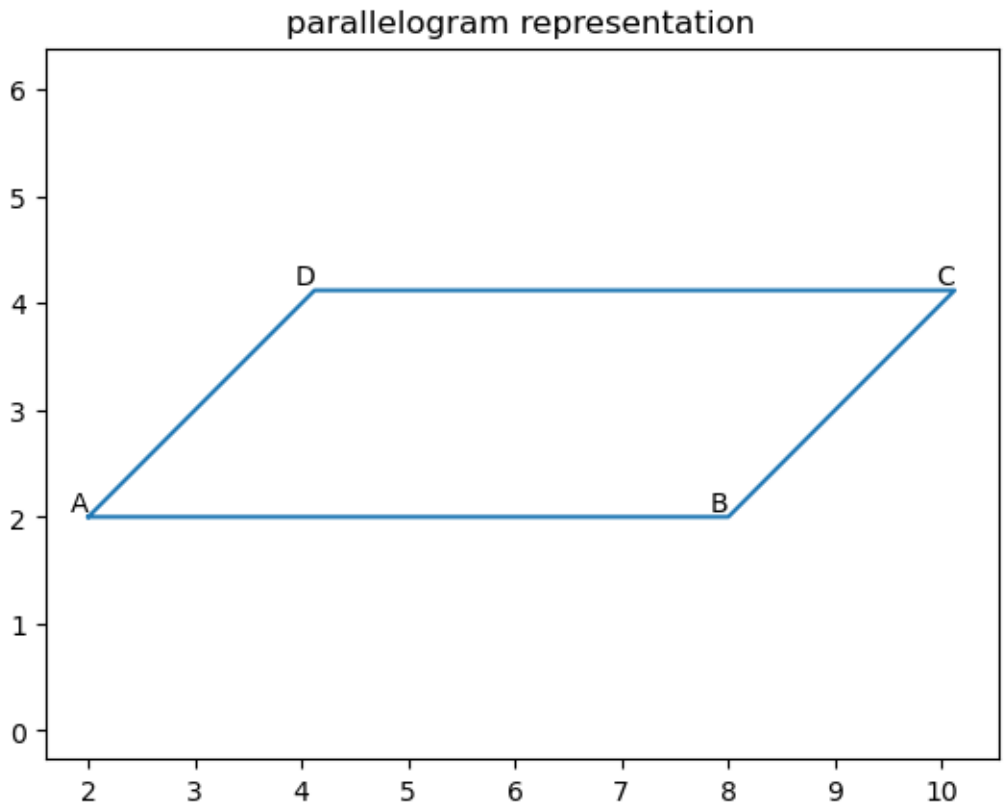
In the following example, we will create a base class called `Shape` and then create a subclass called `Parallelogram` that inherits from `Shape`.

[2]:
```
import matplotlib.pyplot as plt # Import the necessary library for plotting
from math import cos, sin, pi   # Import mathematical functions for cosine,␣
↪sine, and pi

class Shape:
    def __init__(self, x0:float, y0:float):
        self.x0 = x0        # Initialize the x-coordinate of the shape's␣
↪reference point
        self.y0 = y0        # Initialize the y-coordinate of the shape's␣
↪reference point
    def draw_shape(self):   # Define a method to draw the shape, which will be␣
↪overridden in subclasses
        pass                # This method will be overridden in the subclass

# Define a class for parallelograms that inherits from the Shape class,
# since a parallelogram is a specific type of shape with additional properties␣
↪and methods.
class Parallelogram(Shape):

    def __init__(self, x0:float, y0:float, length:float, width:float, angle:␣
↪float):
        super().__init__(x0, y0)            # Call the constructor of the parent␣
↪class
        # Initialize the angle of the parallelogram in radians
        self.angle = angle
        # Initialize the length of the parallelogram
        self.length = length
```

```
        # Initialize the width of the parallelogram
        self.width = width
        # Calculate the height of the parallelogram using the sine of the angle
        self.height= width * sin(angle)
        # Calculate the area (superficy) of the parallelogram using the formula:␣
↪area = base * height
        self.superficy = length * self.height
        # Initialize the coordinates of point A (the reference point)
        self.point_A_x, self.point_A_y = x0, y0
        # Calculate the coordinates of point B by adding the length to the␣
↪x-coordinate of point A
        self.point_B_x, self.point_B_y = x0 + length, y0
        # Calculate the coordinates of point C by adding the length and the␣
↪horizontal component of
        # the width to the x-coordinate of point A, and adding the vertical␣
↪component of
        # the width to the y-coordinate of point A
        self.point_C_x, self.point_C_y = x0 + length + width * cos(angle), y0 +␣
↪width * sin(angle)
        # Calculate the coordinates of point D by adding the horizontal␣
↪component of
        # the width to the x-coordinate of point A, and adding the vertical␣
↪component of
        # the width to the y-coordinate of point A
        self.point_D_x, self.point_D_y = x0 + width * cos(angle), y0 + width *␣
↪sin(angle)


    # Define a method to print the description of the parallelogram,
    # including its properties: length, width, and the starting angle (converted␣
↪to degrees for better readability)
    def description(self):
        print(f"This is a {self.__class__.__name__.lower()} with the following␣
↪properties:")
        print("Length: \t", self.length)
        print("Width: \t\t", self.width)
        # Convert angle from radians to degrees for a more intuitive␣
↪understanding
        print("Angle (in degrees): \t", self.angle * 180 / pi)

    # Define the x-coordinates of the vertices of the parallelogram in order (A,␣
↪B, C, D, and back to A to close the shape)
    def draw_shape(self):
        data_x = [self.point_A_x,
                  self.point_B_x,
                  self.point_C_x,
```

```
                  self.point_D_x,
                  self.point_A_x ]
        data_y = [self.point_A_y,
                  self.point_B_y,
                  self.point_C_y,
                  self.point_D_y,
                  self.point_A_y ]

        labels = ['A', 'B', 'C', 'D']  # Define labels for the vertices of the␣
↪parallelogram
        for i, label in enumerate(labels):
            plt.text(data_x[i], data_y[i], label, fontsize=10, ha='right',␣
↪va='bottom')  # Annotate each vertex with its label

        plt.plot(data_x, data_y, linestyle='-') # Plot the vertices of the␣
↪parallelogram by connecting points A, B, C, D, and back to A
        plt.axis('equal')       # Set equal scaling for x and y axes to maintain␣
↪the shape's proportions
        plt.title(f"{self.__class__.__name__.lower()} representation") # Set the␣
↪title of the plot to the name of the shape
        plt.show()              # Display the plot of the parallelogram
        plt.close()             # Close the plot after displaying it to free up␣
↪resources and avoid displaying multiple plots in interactive environments


    # Define a method to print the area (superficy) of the parallelogram, which␣
↪is calculated in the constructor
    def print_superficy(self):
        print(f"The superficy of the {self.__class__.__name__.lower()} is: {self.
↪superficy}")

parallelogram1 = Parallelogram(2, 2, 6, 3, pi/4)
parallelogram1.description()
parallelogram1.print_superficy()
print(f'the superficy of the {parallelogram1.__class__.__name__.lower()} is:␣
↪{parallelogram1.superficy}')
parallelogram1.draw_shape()
```

```
This is a parallelogram with the following properties:
Length:          6
Width:           3
Angle (in degrees):      45.0
The superficy of the parallelogram is: 12.727922061357857
the superficy of the parallelogram is: 12.727922061357857
```



In this example, we define a base class called `Shape` with an `__init__` method that takes `x` and `y` coordinates as parameters. We also define a `draw_shape` method that is empty and will be overridden in the subclass.
The `Parallelogram` class inherits from `Shape` and has its own `__init__` method that takes additional parameters for width, height, and angle. It also calculates the superficy of the parallelogram. The `draw_shape` method is overridden to provide a specific implementation for drawing a parallelogram using Matplotlib.

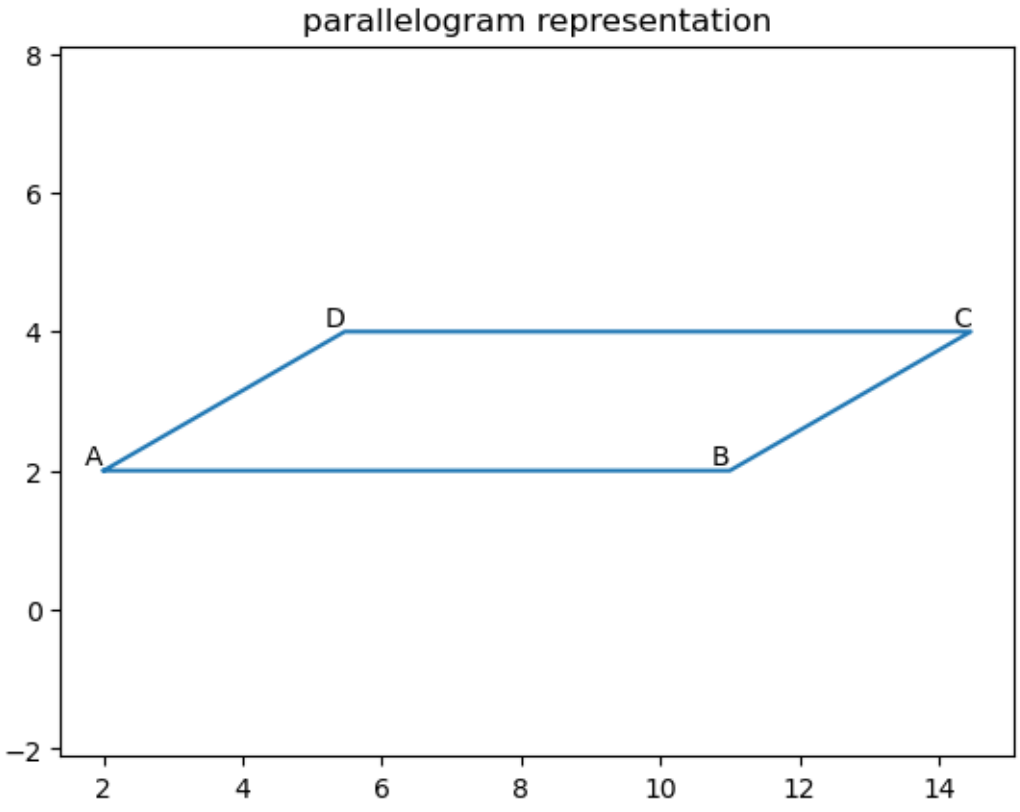
Finally, we create an instance of the `Parallelogram` class and call the `draw_shape` method to visualize it.

[3]:
```
# Define a class for rectangles that inherits from the parallelogram class,
# since a rectangle is a special case of a parallelogram where the angle is␣
↪always 90 degrees (pi/2 radians).
class Rectangle(Parallelogram):
    def __init__(self, x0:float, y0:float, length:float, width:float):
        super().__init__(x0, y0, length, width, pi/2)

    # Define a method to print the description of the rectangle, including its␣
↪properties: length and width
    # (we can omit the angle since it's always 90 degrees for a rectangle)
```

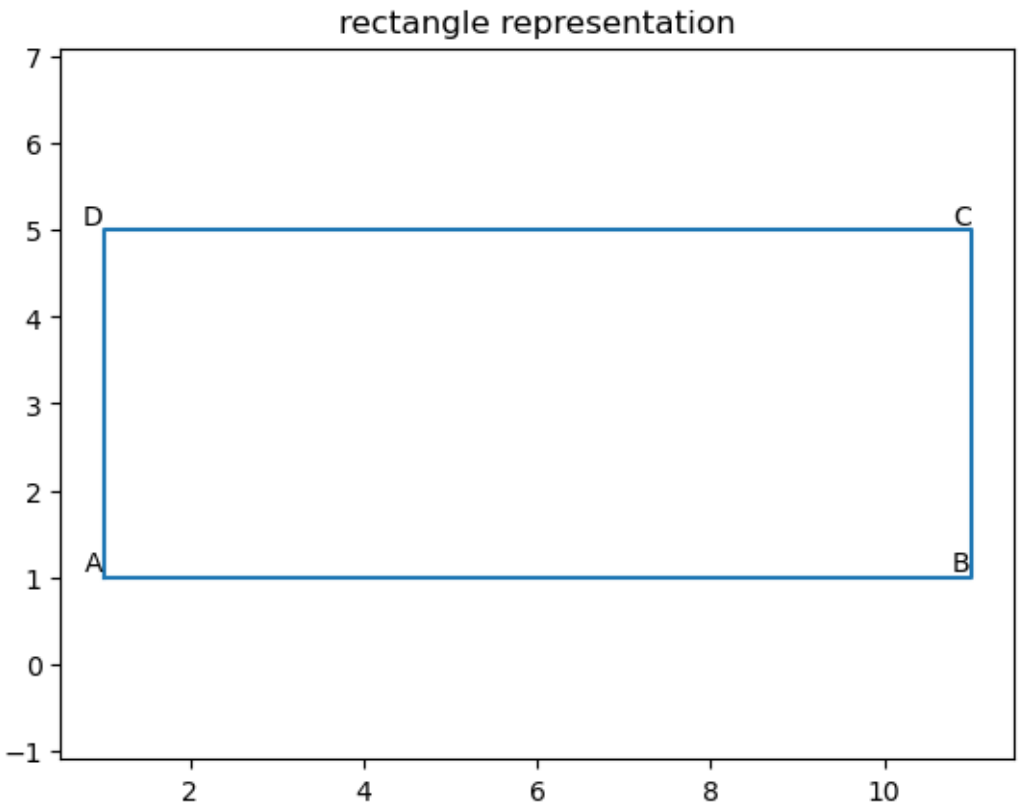
```
    # We can also call the description method of the parent class to reuse the␣
 ↪code for printing length and width,
    # but we will override it here to provide a more specific description for␣
 ↪rectangles.
    def description(self):
        print(f"This is a {self.__class__.__name__.lower()} with the following␣
 ↪properties:")
        print("Length: \t", self.length)
        print("Width: \t\t", self.width)

# Define a class for squares that inherits from the Rectangle class,
# since a square is a special case of a rectangle where the length and width are␣
 ↪equal.
class Square(Rectangle):
    def __init__(self, x0:float, y0:float, side:float):
        super().__init__(x0, y0, side, side)
        self.side=side  # Initialize the side length of the square, which is␣
 ↪equal to both the length and width of the rectangle

    # Define a method to print the description of the square, including its␣
 ↪properties: side length
    # We can also call the description method of the parent class to reuse the␣
 ↪code for printing length and width,
    # but we will override it here to provide a more specific description for␣
 ↪squares.
    def description(self):
        print(f"This is a {self.__class__.__name__.lower()} with the following␣
 ↪properties:")
        print("Side: \t", self.side)


# Create an instance of the parallelogram class and call its methods to␣
 ↪demonstrate functionality
# populating the parameters with some example values (x0=2, y0=2, length=9,␣
 ↪width=4, angle=pi/6 : 30 degrees)
parallelogram1 = Parallelogram(2, 2, 9, 4, pi/6)
parallelogram1.description()
print('the superficy of the parallelogram is:', parallelogram1.superficy)
parallelogram1.draw_shape()

# Create an instance of the Rectangle class and call its methods to demonstrate␣
 ↪functionality
rectangle1 = Rectangle(1, 1, 10, 4)
rectangle1.description()
rectangle1.print_superficy()
print('the superficy of the rectangle is:', rectangle1.superficy)
```

```
rectangle1.draw_shape()

# Create an instance of the Square class and call its methods to demonstrate␣
 ↪functionality
square1 = Square(3, 3, 3)
square1.description()
square1.print_superficy()
print('the superficy of the square is:', square1.superficy)
square1.draw_shape()
```

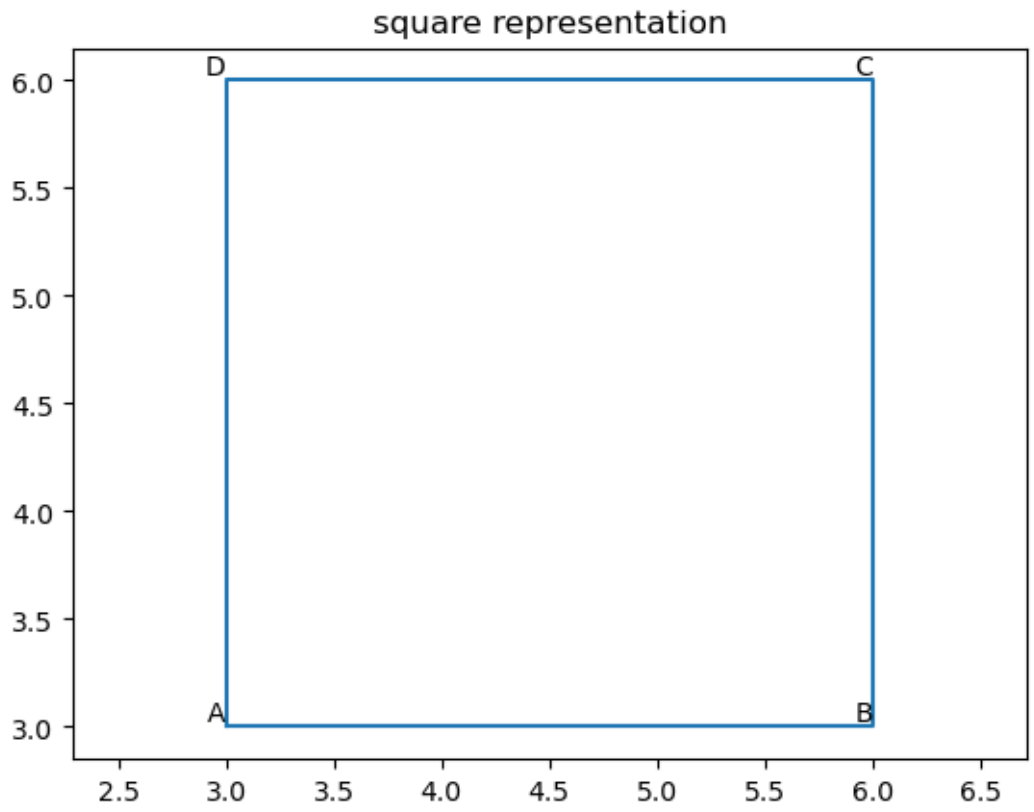
```
This is a parallelogram with the following properties:
Length:          9
Width:           4
Angle (in degrees):      29.999999999999996
the superficy of the parallelogram is: 17.999999999999996
```




```
This is a rectangle with the following properties:
Length:          10
Width:           4
The superficy of the rectangle is: 40.0
the superficy of the rectangle is: 40.0
```




```
This is a square with the following properties:
Side:    3
The superficy of the square is: 9.0
the superficy of the square is: 9.0
```



The description of the code is as follows:

1. We import the necessary libraries: `matplotlib.pyplot` for plotting and `numpy` for numerical operations.
2. We define the `Shape` class with an `__init__` method that initializes the `x` and `y` coordinates and a placeholder method `draw_shape`.
3. We define the `Parallelogram` class that inherits from `Shape`. It has its own `__init__` method that initializes additional attributes for width, height, and angle. It also calculates the superficy of the parallelogram.
4. The `draw_shape` method in the `Parallelogram` class calculates the vertices of the parallelogram based on the given parameters and uses Matplotlib to draw it.
5. The `description` method in the `Parallelogram` class provides a textual description of the parallelogram, including its dimensions and area.
6. we create an instance of the `Parallelogram` class and call the `draw_shape` method to visualize the parallelogram.
7. We define the `Rectangle` class that inherits from `Parallelogram`. It has its own `__init__` method that initializes the length and width of the rectangle, and it overrides the `description` method to provide a more specific description for rectangles.

8. We define the `Square` class that inherits from `Rectangle`. It has its own `__init__` method that initializes the side length of the square, and it overrides the `description` method to provide a more specific description for squares.

9. Finally, we create instances of the `Rectangle` and `Square` classes and call their `description` methods to see the specific details of each shape.

10. The output of the code will include the visualization of the parallelogram and the descriptions of the rectangle and square, demonstrating inheritance and polymorphism in Python OOP.

11. polymorphism is demonstrated through the `description` method, which is overridden in both the `Rectangle` and `Square` classes to provide specific details about each shape, while still being called in a similar way. it is also demonstrated throught the `calculate_area` method, which is defined in the `Parallelogram` class and inherited by both the `Rectangle` and `Square` classes, allowing us to calculate the area of each shape using the same method.